

FULL-STACK APPLICATION DEVELOPMENT

Mobile Application Development Fundamentals

Week 7

React Native • Expo • Mobile Components • API Integration

Instructor: Dilovan Matini

Course Provided By: CAPA & GIZ



Week 6 Recap

TOPICS LEARNED

-  React Router
-  Multi-page Applications
-  Authentication Flow
-  Protected Pages
-  Tailwind CSS

Question

Can we build a mobile application using JavaScript?

This week, we will discover how JavaScript and React can power mobile apps for both Android and iOS.

What Is React Native?

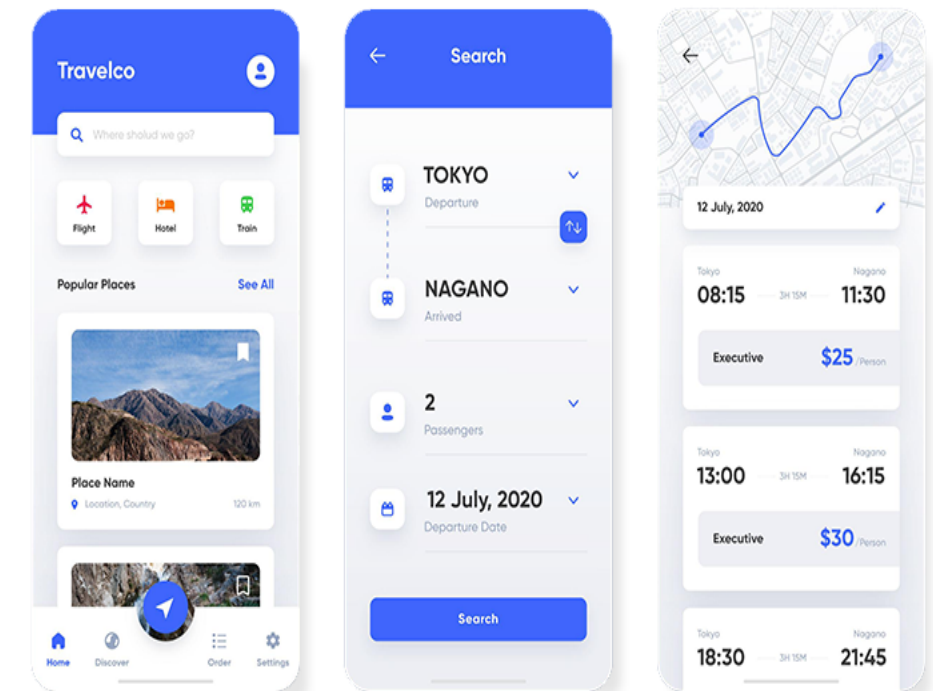
 React Native is a framework used to build **mobile applications** using **JavaScript** and **React**.

Developed by Meta

Build once. Run on:

 **Android**

 **iOS**



Why React Native?

ADVANTAGES

-  JavaScript
-  React Concepts
-  One codebase
-  Native performance
-  Huge community

COMPANIES USING RN

 Facebook

 Instagram

 Discord

 Shopify

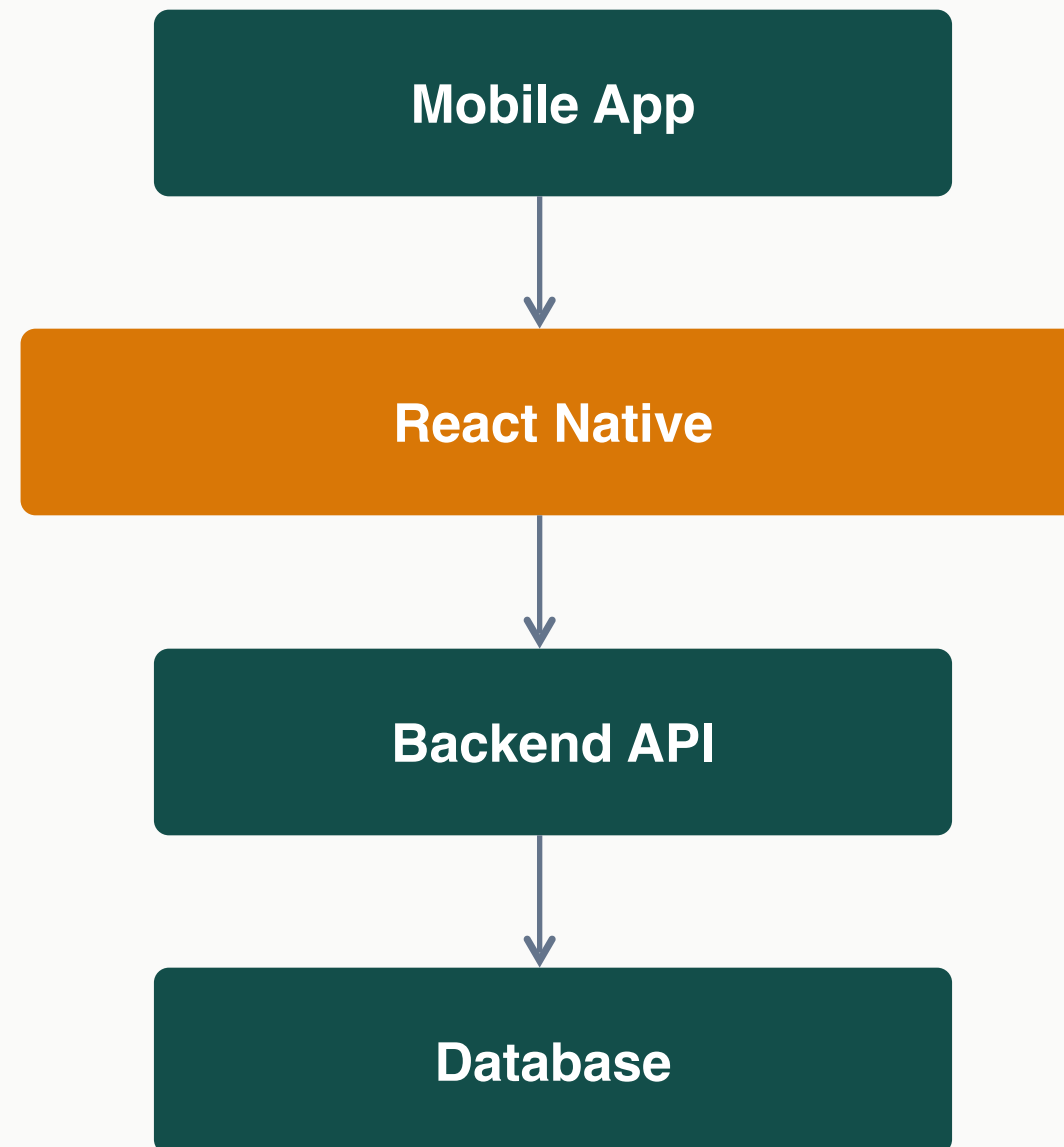


React vs React Native

React	React Native
Websites	Mobile Apps
Browser	Android / iOS
HTML	Native Components

Both share the same React core concepts: components, props, state, and hooks.

Mobile App Architecture



One backend can serve: **Website + Mobile App**

What Is Expo?

Expo is a **development platform** for React Native.

BENEFITS



Easy setup

Get started in minutes



Fast development

Hot reload on save



QR code testing

Test on your phone instantly



No Android Studio

No heavy IDE for basic dev

Installing Expo

Method 1: Global installation

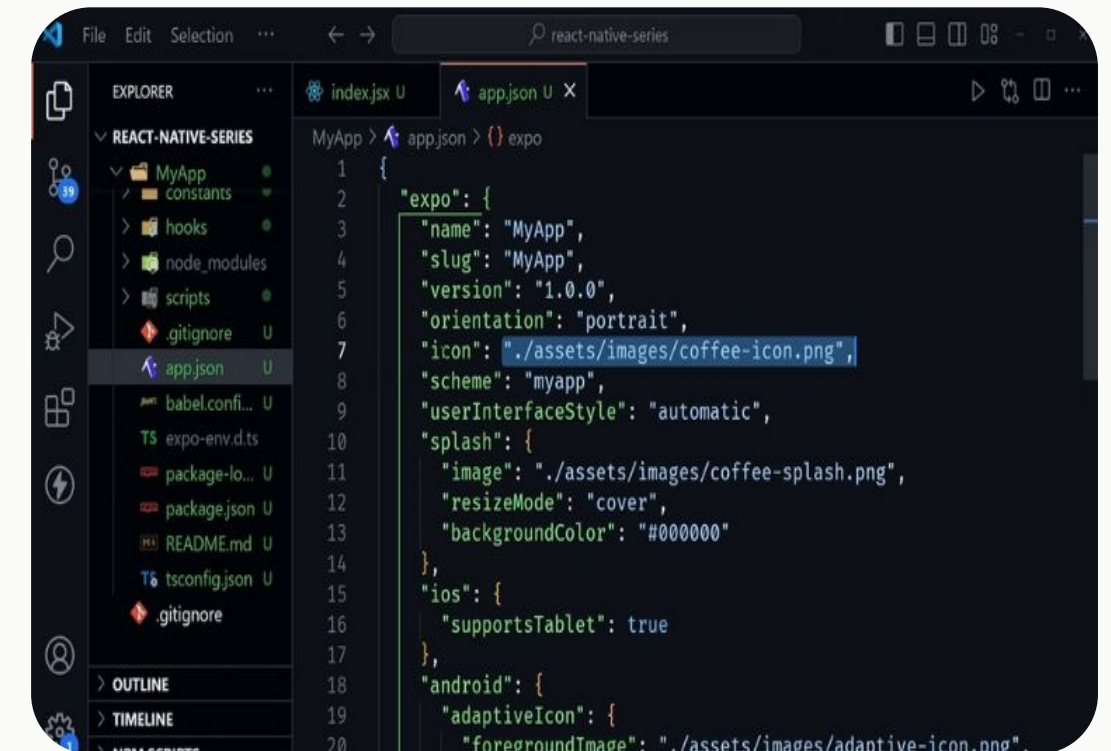
```
npm install -g expo-cli
```

Method 2: Using npx (recommended)

```
npx create-expo-app student-mobile
```



Tip: Method 2 is recommended as it always uses the latest version without global installation.



Creating First Mobile App

1 Create the app

```
npx create-expo-app student-mobile
```

2 Navigate to project

```
cd student-mobile
```

3 Install dependencies

```
npm install
```

4 Start the app

```
npm start
```

WHAT HAPPENS NEXT

Expo will start a development server and display a **QR code** in your terminal.

Scan the QR code with the **Expo Go** app on your phone to see your app live.

Any changes you make to the code will **automatically reload** the app.

Project Structure

student-mobile/

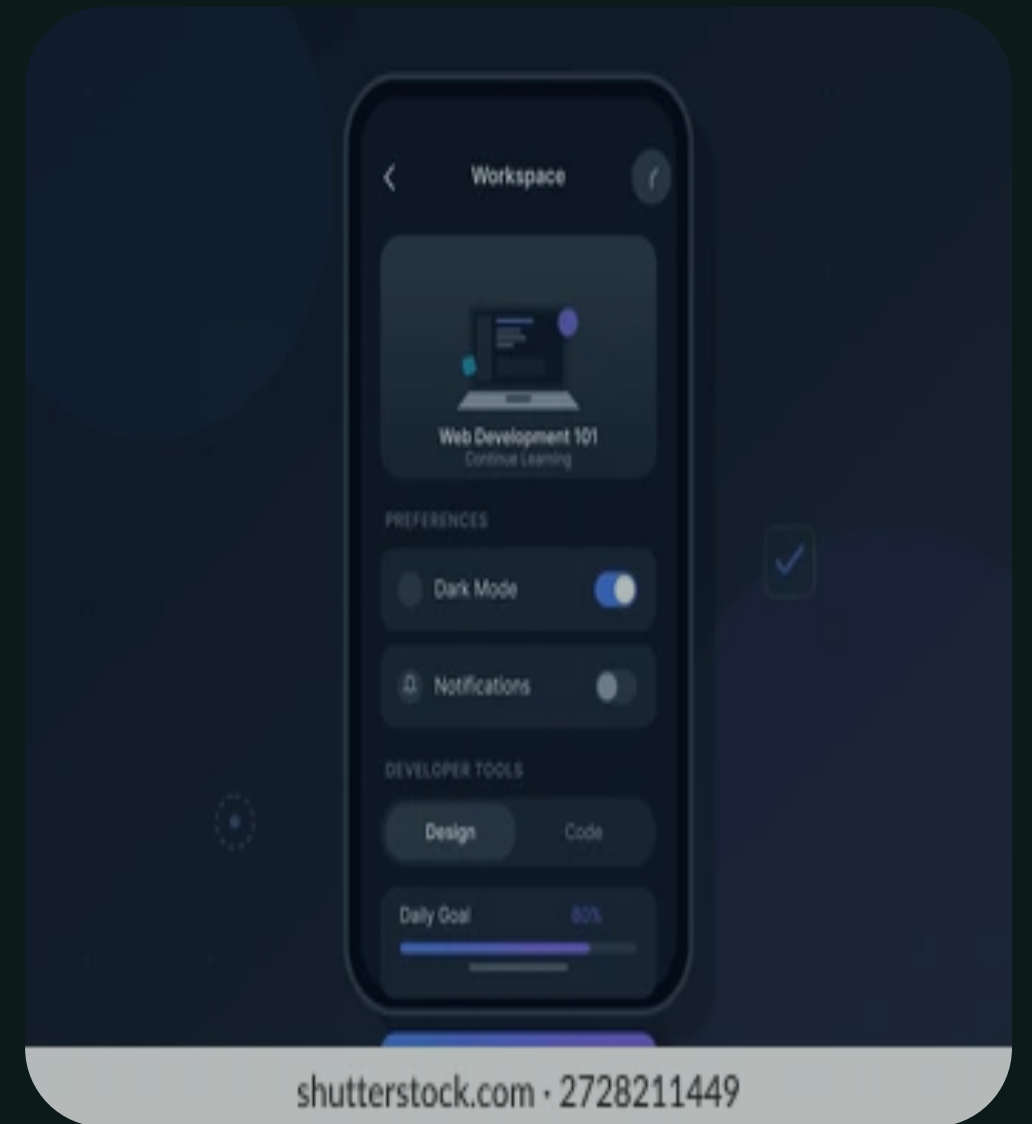
```
|— app/      # Main application code
|— assets/   # Images, fonts, etc.
|— package.json # Dependencies & scripts
└— app.json  # Expo configuration
```

The **app/** folder contains your screens, components, and navigation logic.

Day 2

Mobile Components

View, Text, Image, Button, TextInput, ScrollView



React Native Components

Instead of HTML, use these **Native Components**:



View

Container (like <div>)



Text

Display text (like <p>)



ScrollView

Scrollable container



Button

Touchable button



TextInput

Text input field



Image

Display images



These are the **building blocks** of React Native UI. Each component maps to a native platform element for true native performance.

View Component

REACT NATIVE

```
<View>
  {/* Content goes here */}
</View>
```



Equivalent to

HTML

```
<div>
  <!-- Content goes here -->
</div>
```

Key Points

- The fundamental **container** component
- Used for **layout** and structuring
- Supports **flexbox** styling
- Can be **nested** inside other Views
- Maps to native **UIView** (iOS) or **Android.View**

Text Component

REACT NATIVE

```
<Text>Hello Students</Text>
```



Equivalent to

HTML

```
<h1>Hello Students</h1>
```

Key Points

- Used to display **all text** in the app
- Supports **nesting** for styling
- Can apply **styles** (color, size, weight)
- Must be inside a **View** container

Image Component

USAGE

```
<Image  
  source={{uri:'https://picsum.photos/200'}}  
>
```

Key Points

- Displays images from **remote URL** or **local files**
- Requires **width** and **height** styles
- Use **resizeMode** to control scaling

Preview



Button Component

USAGE

```
<Button  
  title="Login"  
>
```

Key Points

- Creates a **tappable button**
- Requires a **title** prop for the label
- Use **onPress** prop for click handler
- Limited styling — use `TouchableOpacity` for custom buttons

Preview



How the button appears on mobile

TextInput Component

USAGE

```
<TextInput  
  placeholder="Email"  
>
```

Key Points

- Creates a **text input field**
- **placeholder** shows hint text
- Use **onChangeText** to capture input
- Set **keyboardType** for email, number, etc.

Preview

Email

How the input field appears on mobile

Styling in React Native

Instead of Tailwind, use **StyleSheet**:

```
const styles = StyleSheet.create({
  container: {
    padding: 20,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center'
  }
});
```

APPLYING STYLES

```
<View style={styles.container}>...</View>
```

StyleSheet Tips

- Styles are written in **JavaScript objects**, not CSS
- Use **camelCase** (e.g., backgroundColor)
- Numbers default to **dp** (density-independent pixels)
- Strings need **quotes** (e.g., 'center')
- **Flexbox** is the primary layout system
- StyleSheet.create() provides **performance benefits**

Complete Screen Example

Student Profile — combining components:



Image

Student photo using Image component



Name

Student name using Text component



Department

Department info using Text component



Button

Action button using Button component

A practical example combining **Image**, **Text**, **View**, and **Button** components.

ScrollView

Display **long, scrollable content**.

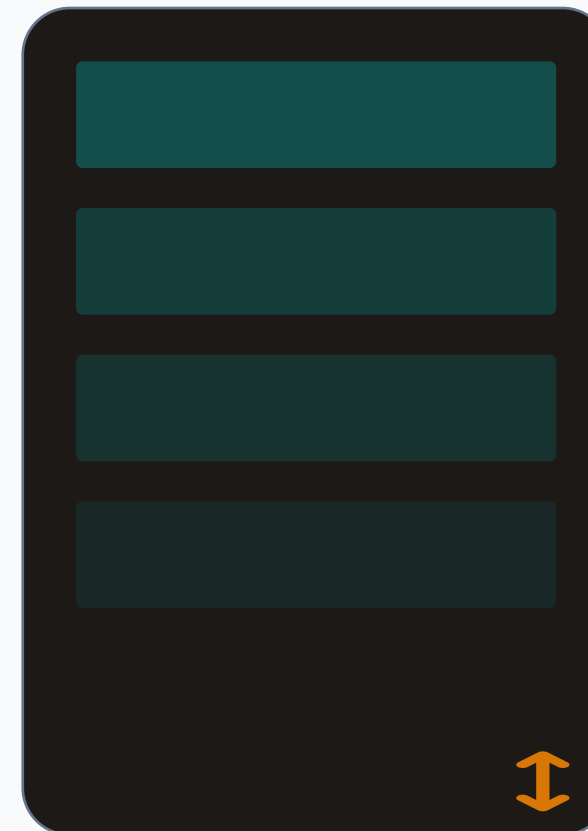
USAGE

```
<ScrollView>
  {/* Long content here */}
</ScrollView>
```

When to Use

- Content **exceeds screen height**
- Long forms or **article content**
- Use **FlatList** for very long lists (better performance)

Visual



Practice Activity

HANDS-ON

Build a Student Profile Screen

CONTAINING



Image



Name



Department



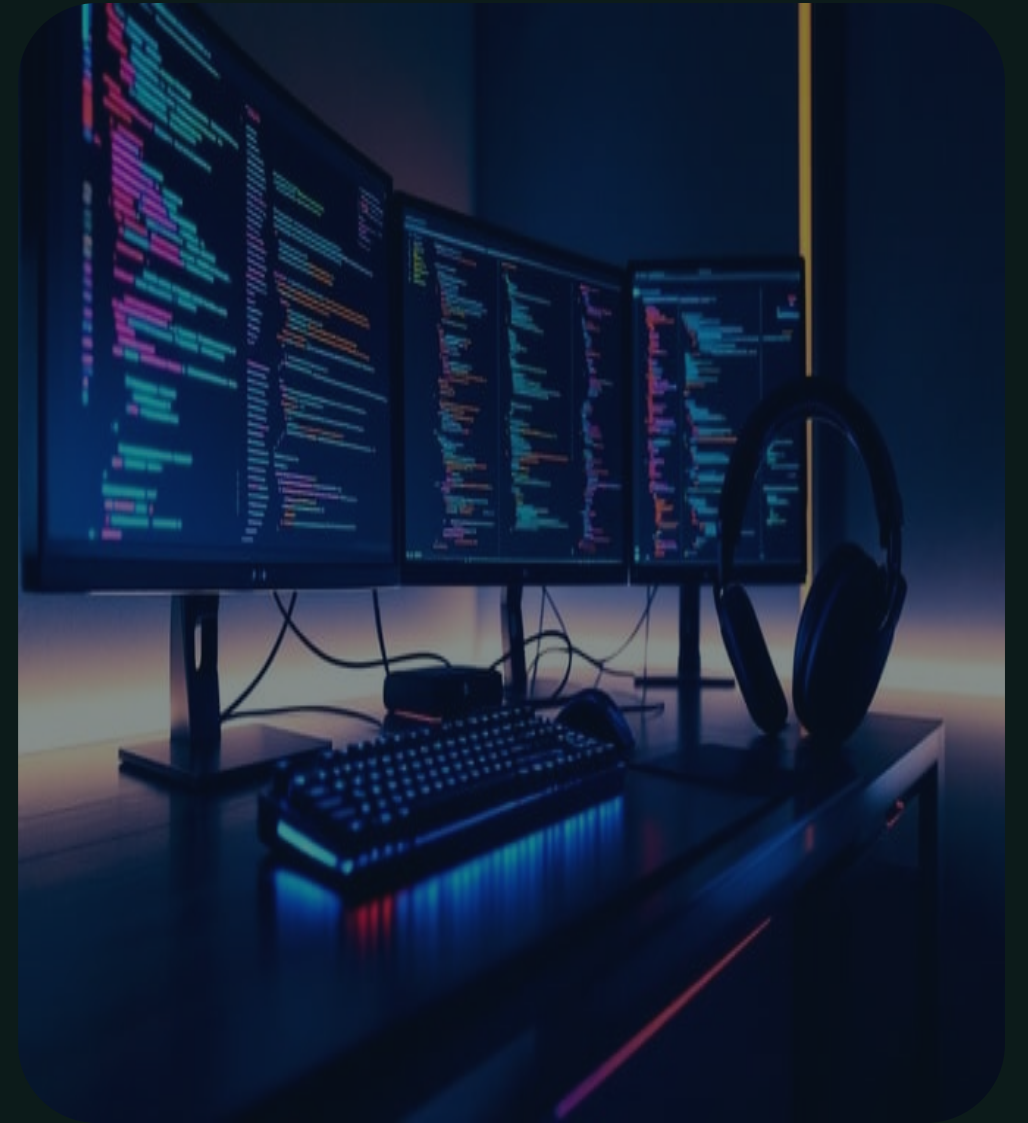
Button

Combine the components you learned to create a complete screen.

Day 3

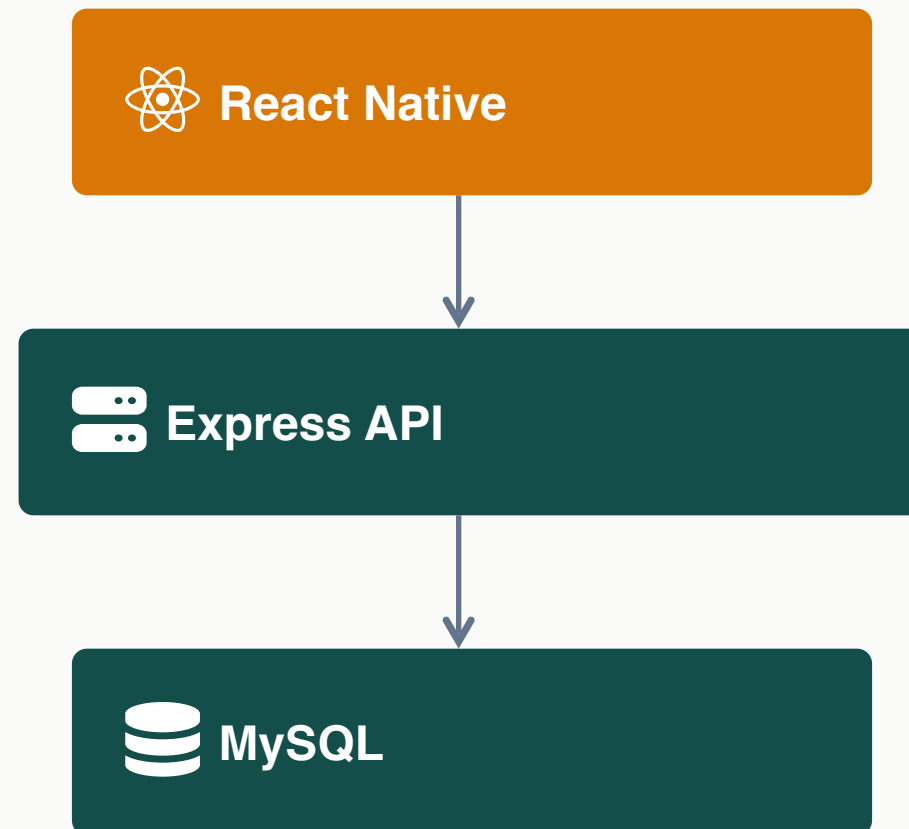
Mobile API Integration

Fetch API, Backend Integration, FlatList



Mobile + Backend

The backend you built in **Week 4** can also serve mobile applications.



Your existing **Express API** and **MySQL database** from Week 4 work seamlessly with your new React Native mobile app.

Fetch Data

Using the **Fetch API** to retrieve data from your backend:

BASIC FETCH

```
fetch("http://YOUR_IP:3000/students")
```

FULL EXAMPLE

```
fetch("http://YOUR_IP:3000/students")
  .then(res => res.json())
  .then(data => {
    console.log(data);
  })
  .catch(err => console.error(err));
```

Important Notes

- Replace **YOUR_IP** with your computer's IP address
- Use **10.0.2.2** for Android emulator
- Use **localhost** for iOS simulator
- Ensure phone and computer are on the **same network**
- Backend must be **running** before fetching

Display Data

Rendering fetched data inside React Native components:

EXAMPLE

```
const [student, setStudent] = useState({});
```

```
<View>
```

```
  <Text>{student.name}</Text>
```

```
</View>
```

How It Works

1. Store fetched data in **state** using `useState`
2. Use `{}` to embed data inside JSX components

Result Preview



John Doe

Computer Science

Data from API

Loading State



Always show feedback while waiting for data — critical for user experience.

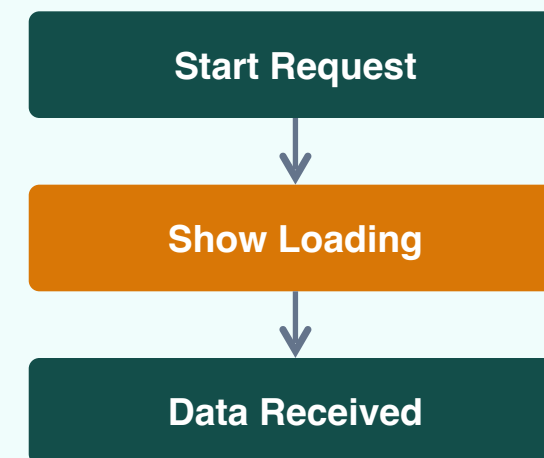
IMPLEMENTATION

```
const [loading, setLoading] = useState(true);

if (loading) {
  return <Text>Loading...</Text>;
}

return <Text>{data}</Text>;
```

FLOW



1. Set loading = true
2. Show spinner or text
3. Fetch data
4. Set loading = false
5. Render data

Common Mobile Components

Essential **component toolkit** for mobile development:



SafeAreaView

Respects device safe zones



View

Container for layout



Text

Display text content



Image

Display images



ScrollView

Scrollable container



FlatList

Efficient list rendering

FlatList is highlighted because it is the most important component for displaying data from your API.

FlatList

Used to display **lists efficiently**. Optimized for large datasets.

USE CASES

 Students

 Products

 Messages

WHY FLATLIST?

- Only renders **visible items** (virtualization)
- Automatic **memory management**
- Built-in **scroll indicator** and **pull-to-refresh**
- Handles **thousands of items** smoothly

BASIC USAGE

```
<FlatList
  data={students}
  renderItem={({item}) => (
    <Text>{item.name}</Text>
  )}
  keyExtractor={item => item.id}
/>
```



Web vs Mobile

Web	Mobile
Mouse	Touch
Browser	App
HTML	Native Components
CSS	StyleSheet

Understanding these differences is key to building great mobile experiences.

Mini Project

PROJECT

Build a Student Mobile App

SCREENS



Home



Student List



Student Details

Fetch students from your **backend API** and display them in the app.

What We Learned



React Native

Framework for building mobile apps with JavaScript



Expo

Development platform for fast React Native setup



Mobile Components

View, Text, Image, Button, TextInput, ScrollView



Styling

StyleSheet.create for mobile-specific styling



Fetch API

Retrieving data from backend services



Backend Integration

Connecting mobile apps to existing APIs

NEXT WEEK

Complete Full-Stack System

Backend → React Website → React Native App → One Database

Instructor: Diloan Matini

Course Provided By: CAPA & GIZ

Thank You